

This is to calculate the GSVA score

```
In [1]: import pandas as pd
import numpy as np
import decoupler as dc
import gseapy as gp

pathF='./temp'
dat=pd.read_csv(f"{pathF}/pseudo_BMSP_twoDis.csv")
dat.head(3)
```

```
/home/yyw9094/.local/lib/python3.9/site-packages/pandas/core/arrays/masked.p
y:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck'
(version '1.3.5' currently installed).
from pandas.core import (
```

```
Out[1]:
```

	Unnamed: 0	AB-24-10	AB-24-11	AB-24-12	AB-24-13	AB-24-14	AB-24-15
0	Mrpl15	89.209006	59.937481	165.542249	0.000000	76.976014	116.55733
1	Lypla1	98.313587	108.012866	335.159409	40.931730	76.829568	157.88964
2	Tcea1	246.485986	89.972230	222.560549	138.603976	191.289161	165.90860

```
In [2]: names=gp.get_library_name()
Mouse=gp.get_library_name(organism='Mouse')
#Mouse
```

```
In [3]: H=gp.get_library(name='MSigDB_Hallmark_2020',
                        organism='Mouse')

### Convert gene sets to decoupler network format
net = []

for pathway, genes in H.items():
    for g in genes:
        net.append((pathway, g, 1))
net = pd.DataFrame(net, columns=["source", "target", "weight"])
net.head(3)
```

```
Out[3]:
```

	source	target	weight
0	TNF-alpha Signaling via NF-kB	MARCKS	1
1	TNF-alpha Signaling via NF-kB	IL23A	1
2	TNF-alpha Signaling via NF-kB	NINJ1	1

```
In [4]: net.source.unique()
```

```
Out[4]: array(['TNF-alpha Signaling via NF-kB', 'Hypoxia',
              'Cholesterol Homeostasis', 'Mitotic Spindle',
              'Wnt-beta Catenin Signaling', 'TGF-beta Signaling',
              'IL-6/JAK/STAT3 Signaling', 'DNA Repair', 'G2-M Checkpoint',
              'Apoptosis', 'Notch Signaling', 'Adipogenesis',
              'Estrogen Response Early', 'Estrogen Response Late',
              'Androgen Response', 'Myogenesis', 'Protein Secretion',
              'Interferon Alpha Response', 'Interferon Gamma Response',
              'Apical Junction', 'Apical Surface', 'Hedgehog Signaling',
              'Complement', 'Unfolded Protein Response',
              'PI3K/AKT/mTOR Signaling', 'mTORC1 Signaling', 'E2F Targets',
              'Myc Targets V1', 'Myc Targets V2',
              'Epithelial Mesenchymal Transition', 'Inflammatory Response',
              'Xenobiotic Metabolism', 'Fatty Acid Metabolism',
              'Oxidative Phosphorylation', 'Glycolysis',
              'Reactive Oxygen Species Pathway', 'p53 Pathway', 'UV Response Up',
              'UV Response Dn', 'Angiogenesis', 'heme Metabolism', 'Coagulation',
              'IL-2/STAT5 Signaling', 'Bile Acid Metabolism', 'Peroxisome',
              'Allograft Rejection', 'Spermatogenesis', 'KRAS Signaling Up',
              'KRAS Signaling Dn', 'Pancreas Beta Cells'], dtype=object)
```

```
In [5]: keywords = 'Inflammatory Response|G2-M Checkpoint'

In_net = net[net['source'].str.contains(keywords, case=False, na=False)]
len(In_net.source.unique())
```

```
Out[5]: 2
```

```
In [6]: dat.set_index(dat.columns[0], inplace=True)
dat
```

Out [6]:

	AB-24-10	AB-24-11	AB-24-12	AB-24-13	AB-24-14	AB-24-15
--	----------	----------	----------	----------	----------	----------

Unnamed:
0

Mrpl15	89.209006	59.937481	165.542249	0.000000	76.976014	116.557330
Lypla1	98.313587	108.012866	335.159409	40.931730	76.829568	157.889646
Tcea1	246.485986	89.972230	222.560549	138.603976	191.289161	165.908606
Atp6v1h	101.373088	140.200297	113.381055	41.172896	127.758928	150.888416
Rb1cc1	178.259327	152.857501	131.118897	124.587207	137.342826	368.809451
...	...	...	...	...	...	...
Cacna1f	0.000000	0.000000	0.000000	0.000000	0.000000	38.564320
F11r	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
Zfp968	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
Gm48065	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
Maff	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

10542 rows × 10 columns

In [7]: In_net

Out [7]:

	source	target	weight
--	--------	--------	--------

1006	G2-M Checkpoint	H2AZ2	1
1007	G2-M Checkpoint	CASP8AP2	1
1008	G2-M Checkpoint	EGF	1
1009	G2-M Checkpoint	H2AZ1	1
1010	G2-M Checkpoint	KNL1	1
...	...	...	...
4443	Inflammatory Response	CCL5	1
4444	Inflammatory Response	PCDH7	1
4445	Inflammatory Response	CCL2	1
4446	Inflammatory Response	ITGB8	1
4447	Inflammatory Response	KCNJ2	1

400 rows × 3 columns

In [8]: dat.index

```
Out[8]: Index(['Mrpl15', 'Lypla1', 'Tcea1', 'Atp6v1h', 'Rb1cc1', '4732440D04Rik',
              'Pcmdt1', 'Rrs1', 'Mybl1', 'Vcpip1',
              ...
              'Etv1', 'Gm17058', 'Ip6k3', 'Dync2li1', 'Celf4', 'Cacna1f', 'F11r',
              'Zfp968', 'Gm48065', 'Maff'],
              dtype='object', name='Unnamed: 0', length=10542)
```

```
In [9]: dat.index = [str(g).upper() for g in dat.index]# [c.capitalize() for c in dat
dat_final = dat.T
dat_final.head(3)
```

```
Out[9]:
```

	MRPL15	LYPLA1	TCEA1	ATP6V1H	RB1CC1	4732440D04RIK
AB-24-10	89.209006	98.313587	246.485986	101.373088	178.259327	32.333423
AB-24-11	59.937481	108.012866	89.972230	140.200297	152.857501	0.000000
AB-24-12	165.542249	335.159409	222.560549	113.381055	131.118897	21.092384

3 rows × 10542 columns

```
In [10]: dcOut=dc.run_gsva(
          mat=dat_final,
          net=In_net,
          source="source",
          target="target"
        )
dcOut
```

```
Out[10]: (source      G2-M Checkpoint      Inflammatory Response
AB-24-10           0.470683           -0.342881
AB-24-11           0.450745           -0.309712
AB-24-12           0.470724           -0.327415
AB-24-13          -0.265852           0.276208
AB-24-14          -0.381139           0.347311
AB-24-15          -0.507698           0.390434
AB-24-16          -0.546894           0.252869
AB-24-17          -0.565599           0.358625
AB-24-8            0.518751          -0.309007
AB-24-9            0.468666          -0.365532,)
```

```
In [11]: import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind

df = dcOut[0].copy()
df['Group'] = ['BM', 'BM', 'BM', 'SP', 'SP', 'SP', 'SP', 'SP', 'BM', 'BM']

#           Day7 Resting
# AB-24-10      0      237
```

```
# AB-24-11 187 0
# AB-24-12 210 0
# AB-24-13 0 51
# AB-24-14 0 114
# AB-24-15 0 181
# AB-24-16 433 0
# AB-24-17 239 0
# AB-24-8 0 119
# AB-24-9 0 202

df['Stage'] = ['Resting', 'D7', 'D7', 'Resting', 'Resting', 'Resting', 'D7', 'D7', 'D7', 'D7']
df
```

```
Out[11]:
```

	source	G2-M Checkpoint	Inflammatory Response	Group	Stage
	AB-24-10	0.470683	-0.342881	BM	Resting
	AB-24-11	0.450745	-0.309712	BM	D7
	AB-24-12	0.470724	-0.327415	BM	D7
	AB-24-13	-0.265852	0.276208	SP	Resting
	AB-24-14	-0.381139	0.347311	SP	Resting
	AB-24-15	-0.507698	0.390434	SP	Resting
	AB-24-16	-0.546894	0.252869	SP	D7
	AB-24-17	-0.565599	0.358625	SP	D7
	AB-24-8	0.518751	-0.309007	BM	Resting
	AB-24-9	0.468666	-0.365532	BM	Resting

```
In [14]: from scipy.stats import ttest_ind

# Define the groups based on your labels
group_bm_Rest = df[(df['Group'] == 'BM') & (df['Stage'] == 'Resting')]
group_sp_Rest = df[(df['Group'] == 'SP') & (df['Stage'] == 'Resting')]
group_bm_D7 = df[(df['Group'] == 'BM') & (df['Stage'] == 'D7')]
group_sp_D7 = df[(df['Group'] == 'SP') & (df['Stage'] == 'D7')]

# Run t-test for each pathway
for pathway in ['G2-M Checkpoint', 'Inflammatory Response']:
    t_stat, p_val = ttest_ind(group_bm_Rest[pathway], group_sp_Rest[pathway])
    t_statD7, p_valD7 = ttest_ind(group_bm_D7[pathway], group_sp_D7[pathway])
    print(f"{pathway} P-value: {p_val:.4f}; p_D7: {p_valD7:.4f}")
```

G2-M Checkpoint P-value: 0.0003; p_D7: 0.0002
Inflammatory Response P-value: 0.0001; p_D7: 0.0073

```
In [22]: # # Melt the dataframe to long format for easier plotting with Seaborn
# df_melt = df.melt(id_vars='Group', var_name='Pathway', value_name='GSVA Score')

# # Set up the plot
# plt.figure(figsize=(10, 6))
```

```

# sns.boxplot(data=df_melt, x='Pathway', y='GSVA Score', hue='Group', palette=
# sns.stripplot(data=df_melt, x='Pathway', y='GSVA Score', hue='Group', dodge

# plt.title('GSVA Score Comparison by Group')
# plt.legend(title='Group', loc='upper right')
# plt.show()

df_rest=df[df['Stage'] == 'Resting'].drop('Stage', axis=1)
df_rest

df_D7=df[df['Stage'] == 'D7'].drop('Stage', axis=1)
df_D7

```

Out[22]:

source	G2-M Checkpoint	Inflammatory Response	Group
AB-24-11	0.450745	-0.309712	BM
AB-24-12	0.470724	-0.327415	BM
AB-24-16	-0.546894	0.252869	SP
AB-24-17	-0.565599	0.358625	SP

In [30]:

```

import seaborn as sns
import matplotlib.pyplot as plt

plt.rcParams['pdf.fonttype'] = 42

my_colors = {'BM': 'grey', 'SP': 'blue'}

# 1. Reshape data for plotting
df_long = df_rest.melt(id_vars='Group',
                        var_name='Pathway',
                        value_name='Score')

df_long

## 2. Create the plot
plt.figure(figsize=(5, 6))

# Boxplot (the boxes)
sns.boxplot(data=df_long,
            x='Pathway',
            y='Score',
            hue='Group',
            palette=my_colors,
            showfliers=False)

# Stripplot (the jittered points)
sns.stripplot(data=df_long, x='Pathway', y='Score', hue='Group',
              dodge=True, color='black', alpha=0.6, jitter=True, size=10)

# Clean up legend (to avoid double entries from box/strip)
handles, labels = plt.gca().get_legend_handles_labels()
plt.legend(handles[0:2], labels[0:2], title='Group', bbox_to_anchor=(1.05, 1

```

```
plt.title('GSVA Scores: BM vs SP')
plt.tight_layout()

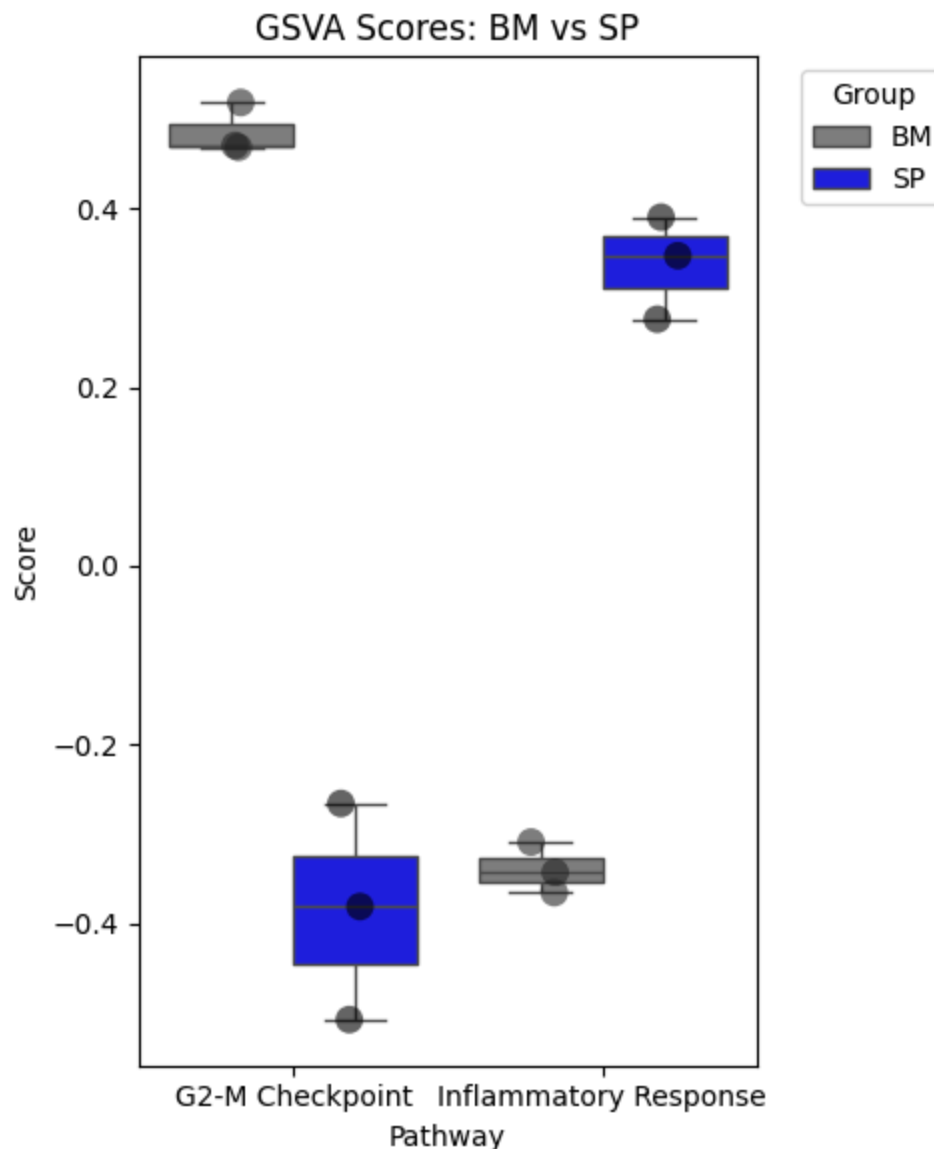
plt.savefig(f'{pathF}/GSVA_Scores_rest.pdf', format='pdf', bbox_inches='tight')

plt.show()
```

/tmp/ipykernel_3569412/2286154736.py:28: FutureWarning:

Setting a gradient palette using color= is deprecated and will be removed in v0.14.0. Set `palette='dark:black'` for the same effect.

```
sns.stripplot(data=df_long, x='Pathway', y='Score', hue='Group',
```



```
In [31]: plt.rcParams['pdf.fonttype'] = 42
# 1. Reshape data for plotting
df_long2 = df_D7.melt(id_vars='Group',
                      var_name='Pathway',
                      value_name='Score')
```

```

df_long2

## 2. Create the plot
plt.figure(figsize=(5, 6))

# Boxplot (the boxes)
sns.boxplot(data=df_long2,
            x='Pathway',
            y='Score',
            hue='Group',
            palette=my_colors,
            showfliers=False)

# Stripplot (the jittered points)
sns.stripplot(data=df_long2, x='Pathway', y='Score', hue='Group',
             dodge=True, color='black', alpha=0.6, jitter=True, size=10)

# Clean up legend (to avoid double entries from box/strip)
handles, labels = plt.gca().get_legend_handles_labels()
plt.legend(handles[0:2], labels[0:2], title='Group', bbox_to_anchor=(1.05, 1))

plt.title('GSVA Scores: BM vs SP')
plt.tight_layout()

plt.savefig(f'{pathF}/GSVA_Scores_D7.pdf', format='pdf', bbox_inches='tight')

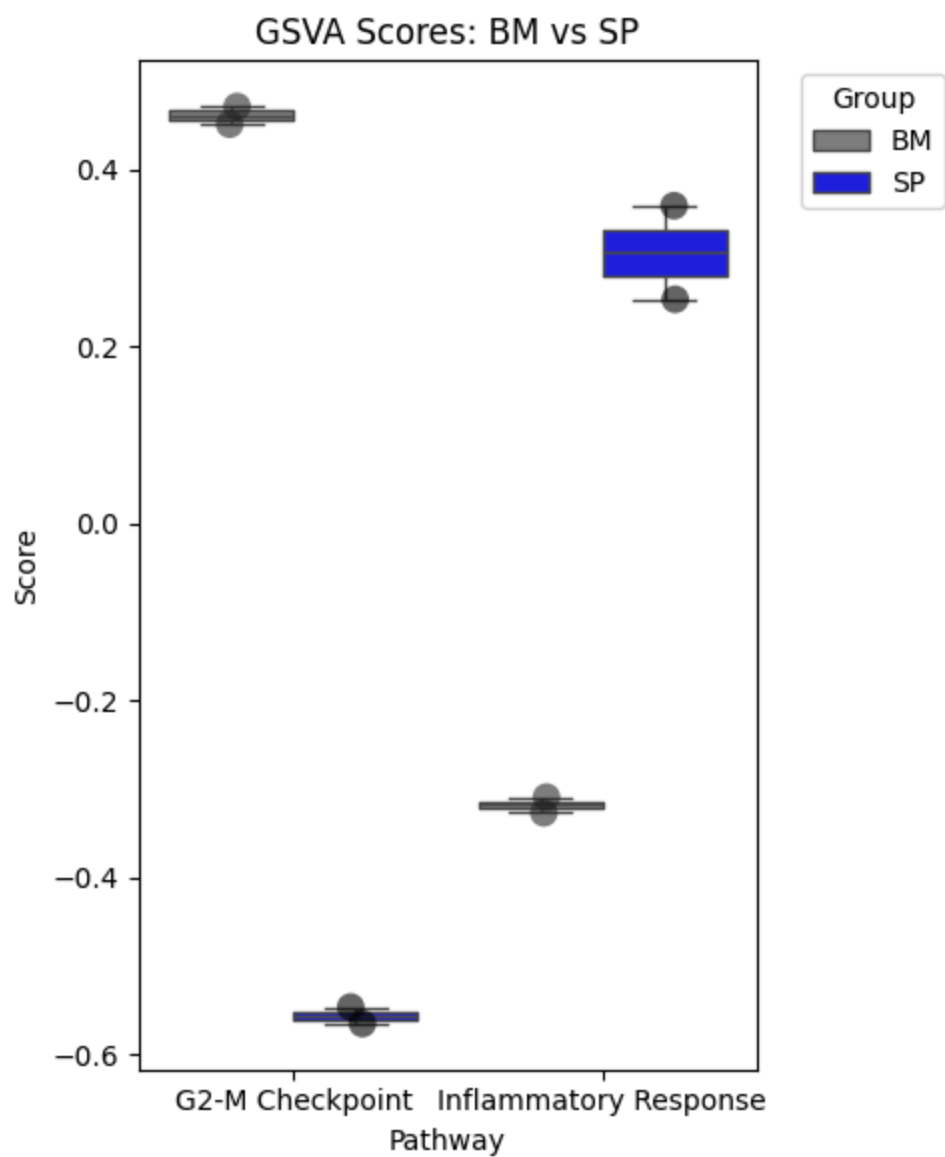
plt.show()

```

/tmp/ipykernel_3569412/3611028331.py:22: FutureWarning:

Setting a gradient palette using color= is deprecated and will be removed in v0.14.0. Set `palette='dark:black'` for the same effect.

```
sns.stripplot(data=df_long2, x='Pathway', y='Score', hue='Group',
```

In []: